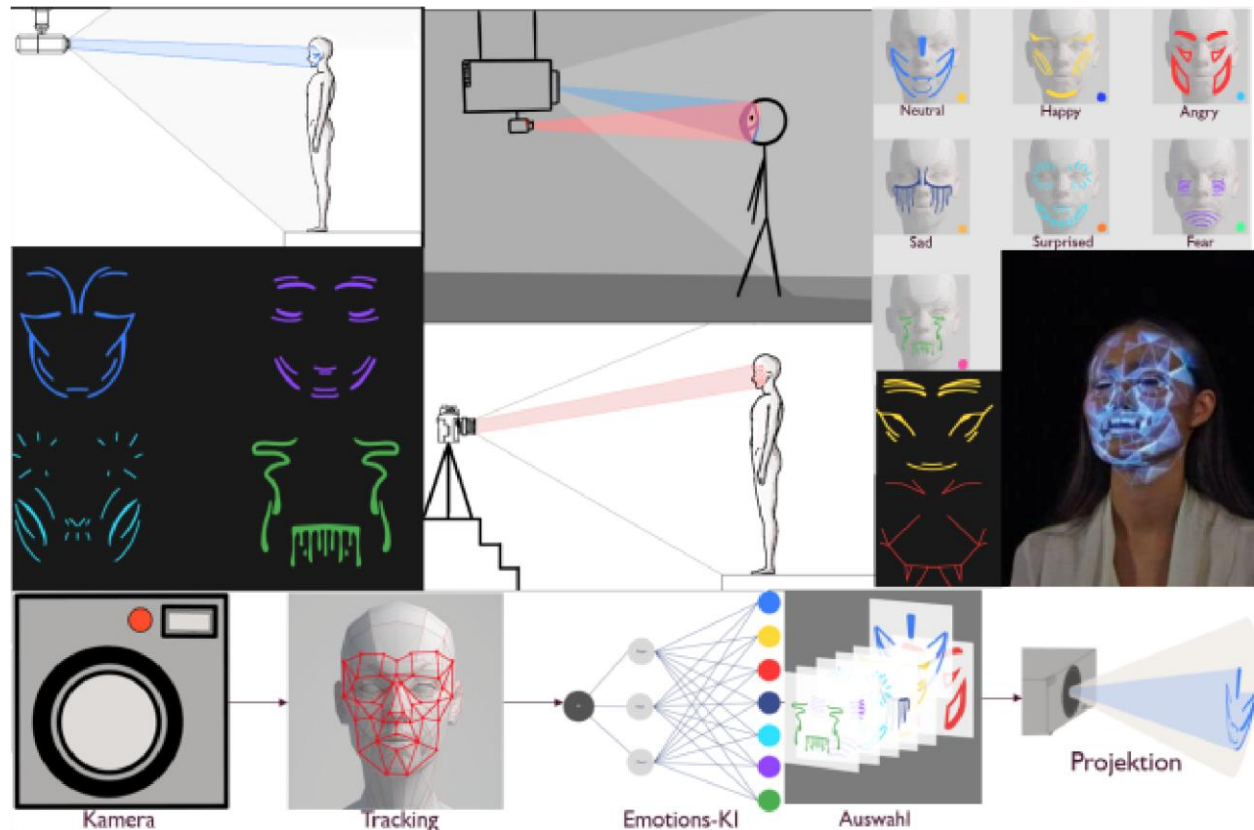




EarlGrey - Face-Projection

22.12.2025

von:

Robert Tschorn, Lina S.

Dozent: Lars E.

IXLAB | Fakultät Informatik / TU Dresden

Sommersemester 2025, Wintersemester 25/26

Konzept & Ideen Entwicklungspfad

Anfangsidee

Konzept

Eine Kamera erfasst eine Person und generiert daraus ein Lichtbild-Avatar: eine ästhetische Kombination aus Formen, Farben und Bewegungen, die Emotionen/ den Vibe der Person visuell repräsentativ darstellt.

Primärer Output

DMX Licht: Avatar wird in den Raum, in Nebel ausgestrahlt.

Visualisierung

Reduzierte, klare Formen (Kreise, Linien, Vierecke, Partikel), Farbpalette x Emotion, leichte Bewegungsmuster (Puls, Wellen, Drehen, Flimmern, etc.).

Update-Takt

Kontinuierlich in Echtzeit

Annahmen/ Grund für Idee

DMX-Hardware verfügbar, Nebelmaschine verfügbar, Raumlicht dimmbar.

Überarbeitung

Neues Konzept

Beamer-Projektion auf Gesicht (Maske) und Schultern (Aura).

- Maske: stilisierte PNG-Overlays mit Transparenz (Formen, Texturen, Farben, Glow) direkt aufs Gesicht
- Aura: Overlay auf Schultern in der Komplementärfarbe der Maske, form passend zur Emotion
- Update: alle 15s für mehr Stabilität, ruhigeres Bild, besser kontrollierbar

KI-Komponente

KI entscheidet „alles“ im Sinne von:

- Emotion → Maske → Farbpalette → Schultern/ Bewegung
- *Optionale Funktion:* Gegenstandserkennung als Trigger (Lichtschwert erkannt → DMX um Stimmung zu Ändern)

Warum die Änderungen?

DMX nicht wirklich in der Lage präzise Formen wie gewünscht auszustrahlen, Projektion passt da besser, ist präziser, einfacher veränderbar, liefert einen besseren „Glow“ Effekt

Projektbeschreibung (Zwischenstand)

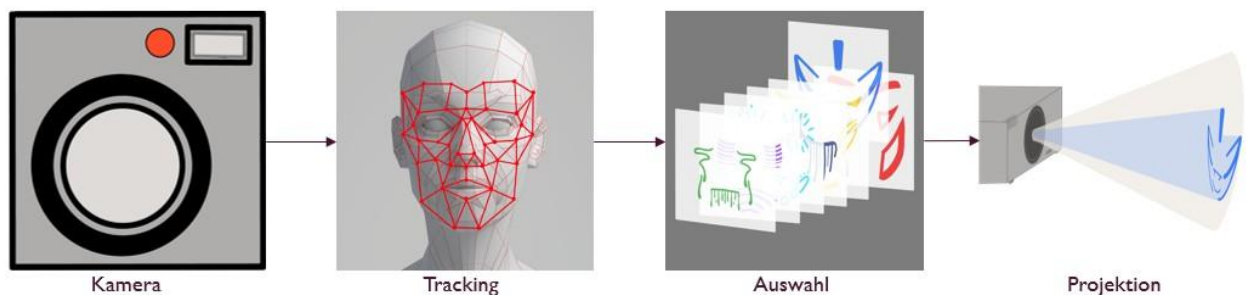


Abbildung 1: Pipeline

Ziel

Emotionen und Zustände visuell auf Personen sichtbar machen: Eine Kamera erkennt das Gesicht, eine KI wählt eine Maske (Form/Farbe) und projiziert sie direkt auf das Gesicht.

Warum So?

1. Feedback ist direkt und körperbezogen statt abstrakt auf einer Leinwand/Oberfläche
2. Künstlerisch steuerbar: Masken-Design + Farbpaletten repräsentieren Emotion/„Vibe“

Besonderheiten

- 2-Fenster-Pipeline (ein Preview-Fenster und ein direktes Ausgabefenster)
- PNG-Masken mit Alphakanal (Transparenz)
- Modular erweiterbar (Mensch wählt aus → KI steuert Auswahl, usw.)

Konzeption

Mapping

Emotion → Maske → Farbe → (*optional*) Schulter-Akzent-farbe

Beispiel: „Wütend“ → scharfe, deutlich erkennbare Kanten → Rot → (*Türkis als Komplementär*)

Räume

- Image-Space: Kamerabild (Capture-Auflösung)
- Window-Space: Preview- & Projektor-Fenstergrößen
- Projection-Space: Gesicht (& *evtl. Schultern*) einer realen Person (genaue Kalibrierung Beamer → Gesicht folgt)

Zusammenfassung

Ursprünglich startete das Projekt als Lichtbild-Avatar mit DMX-Steuerung und Nebel. Aufgrund mangelnder Präzision von DMX wurde das Konzept überarbeitet, hin zu einer direkten Projektion auf das Gesicht (Face-Mapping) mittels Beamer. Eine KI soll die Emotionen des Gesichts erkennen und passende Masken projizieren.

Umsetzung

Umsetzung (Idee)

1. Tracking + Projektion getrennt stabil bekommen
2. Masken & Farbpaletten ausarbeiten (je Emotion verschiedene Varianten, Komplementärfarben definieren)
3. Tracking & Projektion zusammenführen (Maske auf Gesicht)
4. Emotions-KI implementieren (Lokales Modell)
5. Kalibrierung (Kamera ↔ Beamer)
6. *(optional) Form/ Aura in Komplementärfarbe auf Schultern*

1. Versuch (Python)

Architektur

- Tracking: MediaPipe/ Face-Landmarks → Gesichtsmitte + Bounding-Box
 - Masken: PNG mit Alpha; zentrale Ausrichtung; skalieren/ positionieren anhand der Tracking Daten
 - Ausgabe: Preview = Kamera-Aufnahme + Maske; Projektion = nur Maske auf schwarzem Hintergrund
- Projektaufbau**
- assets/masks/ - Masken Bibliothek (PNG/Alpha, 1000–1500 px Breite)
 - camera/tracker.py - Webcam-Capture + Gesichts-/Schulter-Tracking
 - visuals/avatar_generator.py - Masken-/ Farblogik, Avatar Generierung
 - main.py - Zusammenführen der Module, Fenstererzeugung

Pipeline

Kamerainput → Landmark-Tracking

- Preview: Kamerabild + halbtransparentes Masken-Overlay
- Beamer: nur Maske, schwarzer Hintergrund, Maskengröße skaliert auf Gesicht; Position = Gesichtszentrum

Ergebnis

- stabiles, schnelles Tracking; Ausgabequalität gut, könnte besser sein (scharfe Kanten)

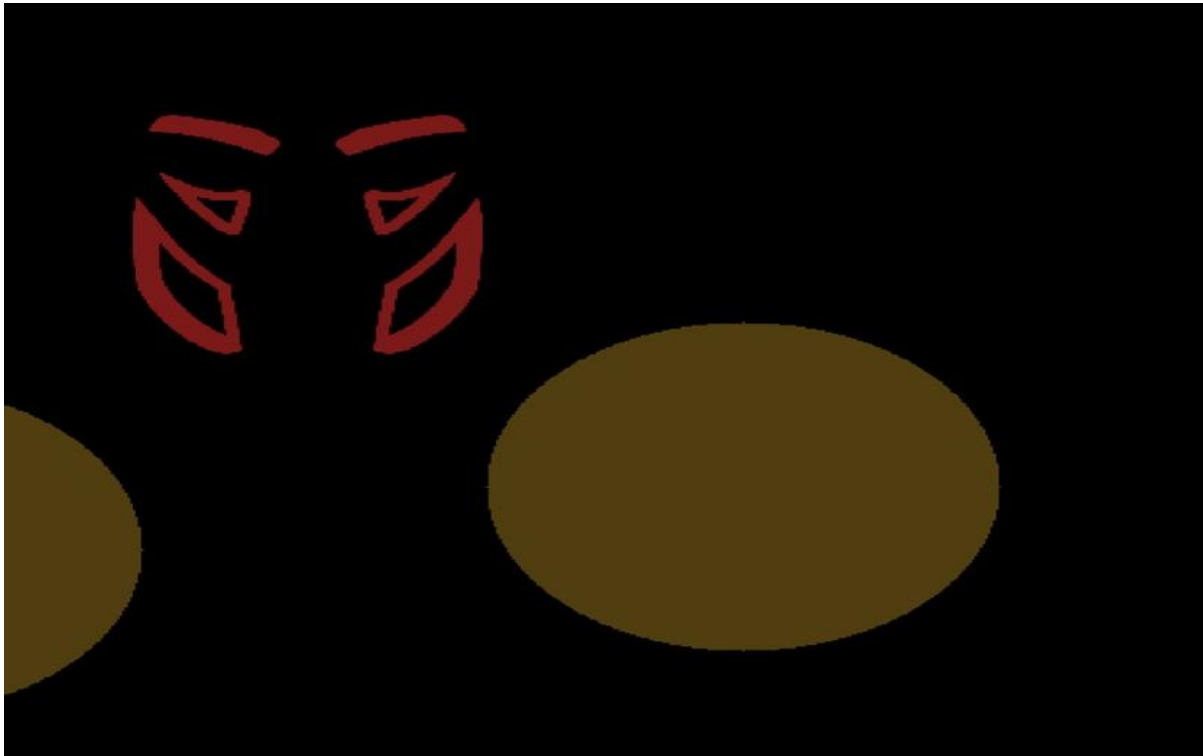


Abbildung 2: Ausgabe Python Version

2. Versuch (C++) [Cinder, OpenCV]

Architektur

- App-Layer: Zwei Fenster, Draw-Routing, Hotkeys zum Umschalten (Masken, Preview
↔ Projektor)
- Preview-Layer: Kamera-Capture, Tracking, Debug-Draw
- Projektor-Layer: Masken-Rendering (PNG/Alpha) deckungsgleich auf Gesicht

Projektaufbau (Kern)

- EarlGreyApp.cpp - Fenster/ Signale, Hotkeys
- FacePreview.cpp - Kamera- Bild + Maske
- FaceTracker.cpp - Gesichtstracking & Erkennung, Jittering-Glättung
- ProjectorView.cpp - Maske + schwarzer Hintergrund + Shader für Maske

Pipeline (simplified)

Kamera → Tracking → Skalierung → Positionierung → Ausgabe

Ergebnis

- (Noch) anfällig für Jittering; unverlässlich bei Brillen; Masken Qualität gut, je nach Auflösung; gute Basis, definitiv ausbaubar
- Maske wird nicht statisch projiziert sondern je nach Position und Zeitpunkt variierend

Zusammenfassung

Das Ziel ist es, Emotionen visuell sichtbar zu machen. Eine Kamera erfasst das Gesicht, eine KI wählt eine Maske (Form/Farbe) und projiziert sie direkt zurück auf das Gesicht. Die Pipeline besteht aus zwei Fenstern (Preview & Projektor) und nutzt PNG-Masken mit Transparenz.

Projektübersicht (Finaler Stand)

FaceProjection ist eine interaktive Installation, die wie eine Art „digitaler Spiegel“ fungiert. Sie erfasst das Gesicht des Nutzers in Echtzeit, analysiert dessen emotionalen Zustand mittels moderner KI und projiziert passend dazu digitale Masken direkt auf das Gesicht.

Kern-Features

- **Tracking:** Einsatz von OpenCV YuNet (KI-Modell) für stabiles Gesichtstracking, auch bei Bewegungen
- **Emotionserkennung:** Ein FER+ Modell (Facial Emotion Recognition) analysiert 8 verschiedene Emotionen (*Neutral, Happiness, Surprise, Sadness, Anger, Disgust, Fear, Contempt*) in Echtzeit
- **Hierarchische-Emotions-Logik:** Eine komplexe Prioritäten-Steuerung verhindert, dass ein leichtes Lächeln starke, aber seltene Emotionen wie „Disgust“ oder „Contempt“ überschreibt. Eine häufige Emotion wie „Happiness“ gewinnt nur über eine Seltene Emotion, wenn sie signifikant stärker ausgeprägt ist
- **Visuelle Effekte:** Ein custom GLSL-Shader sorgt für ein organisches, an Wellen erinnerndes „wobbeln“ der Masken, synchronisiert zwischen Vorschau- und Beamer-Ansicht
- **Stabilität:** Implementierung von „Debouncing“ (Entprellen) und Glättungsfaktoren, um das Flackern der Masken zu verhindern.

Umsetzung

Architektur

- Ressource Management: Einführung strikter „Scoped-Blocks“ für Shader und Texturen, um Grafikfehler und Abstürze zu verhindern.
- Hierarchische Logik: Implementierung einer Gewichtungs-Logik im Tracker, die seltenen und häufigen Emotionen verschiedene Prioritäten zuweist, da Emotionen wie Implementierung einer Gewichtungs-Logik im Tracker, Implementierung einer Gewichtungs-Logik im Tracker, da Emotionen wie „Contempt“ weniger Trainingsdaten im FER+ Modell haben und somit seltener ausschlagen

Projektaufbau

- FaceTracker.cpp: YuNet + FER+ Integration mit „Debouncing“ (Entprellen) und Emotionen-Prioritäten-Logik (Schwellenwerte).
- ProjectorView.cpp: Sicheres Shader-Rendering, Synchronisation der Shader-Zeit (elapsed) mit der Preview, manuelle Offset-Steuerung der Maske.
- FacePreview.cpp: Visualisierung der getrackten Daten + Debugging.

- EarlGreyApp.cpp: Bereinigte Steuerung (Entfernung fehlerhafter Modi), neuer „Force-Mode“ (Taste S) für Demos.

Pipeline

Kamera Input → YuNet (FaceTracking) → FER+ (Emotionserkennung) →

Priorisierungs-Algorithmus → GLSL Shader-Rendering (Bewegung) → Projektion

Ergebnis

- Stabilität: Keine Abstürze, wenig jittering
- Qualität: Organisches „Atmen“ der Masken durch synchronisierte Shader; präzisere Emotionsauswahl durch die neue Gewichtungslogik.

How-To Setup

Voraussetzungen

- Hardware
 - Webcam/Kamera mit guter Qualität für besseres Tracking
 - Beamer (oder 2. Bildschirm zum testen)
 - Stabile Halterung
- System
 - Windows 10/11 (getestet), Grafiktreiber (aktuell)
 - Erweiterter Desktop aktiviert
- Umgebung
 - Gesicht muss gut beleuchtet sein für zuverlässiges Tracking

Start

1. Beamer und Kamera anschließen und Beides auf Person ausrichten
2. App starten (Release-Build)
 - Zwei Fenster öffnen sich
 - a. Preview (links)
 - b. Projektor (rechts)
3. Projektor Fenster auf 2. Ausgabegerät ziehen und **F** für Vollbild drücken
4. Maske per Pfeiltasten auf das Gesicht verschieben und die Größe per **,/.** anpassen
5. Im Projektor Fenster: ProjektorView Ansicht mit **d** umschalten zu FacePreview
6. zum beenden 2 mal **ESC** drücken

Steuern (Hotkeys)

- **F** Vollbild/ Window im aktiven Fenster umschalten
- **D** Projektor Fenster zwischen Maske und Preview umschalten
- **Pfeiltasten** Maske verschieben (Offset X/Y)
- **,/.** Maske vergrößern / verkleinern (Zoom)
- **1/2/3** Fensterauflösung ändern (1080p / 720p / 480p)
- **ESC** Aktives Fenster schließen

Troubleshooting

Problem	Datei & Ort	Fix
Falsche Kamera wird genutzt	FacePreview.cpp → setup()	Capture::create(..., devices[0]) Ändere [0] zu [1] oder [2], um eine andere Kamera zu wählen, bis es die gewünschte ist.
Kamera-Auflösung ändern	FacePreview.cpp → setup()	Capture::create(1920, 1080, ...) Zahlen anpassen (z.B. 1280, 720).
Maske ist viel zu groß/klein	ProjectorView.hpp	float mScaleMult = 2.0f; Ändere den Startwert (z.B. auf 1.5f für kleiner oder 3.0f für größer).
Emotions-Erkennung zu schnell	FaceTracker.hpp	const int EMOTION_THRESHOLD_FRAMES = 8; Erhöhe den Wert (z.B. auf 15), damit Emotionen länger gehalten werden müssen, bevor die Maske wechselt.
Bestimmte Emotion wird nie erkannt	FaceTracker.cpp → update()	Schwellenwerte (Thresholds) in der Logik anpassen (z.B. if (mCurrentScores[...]) > 0.15f) senken auf 0.10f .

Ausblick & Erweiterungsmöglichkeiten

- **Automatische Kalibrierung Kamera ↔ Beamer:** Implementierung einer Routine/ Logik, die Beamer und Kamera automatisch aufeinander abstimmt, sodass manuelles Verschieben entfällt.
- **Generative KI:** Statt statischer PNGs könnte eine KI Masken live generieren, um noch individueller auf Emotionen einzugehen.
- **Multi-User Support:** Erweiterung des Trackings auf mehrere Gesichter gleichzeitig für Interaktionen zwischen Personen.
- **DMX-Integration:** Wiederaufnahme der Lichtsteuerung, um den Raum passend zur Emotion farblich zu tauchen.

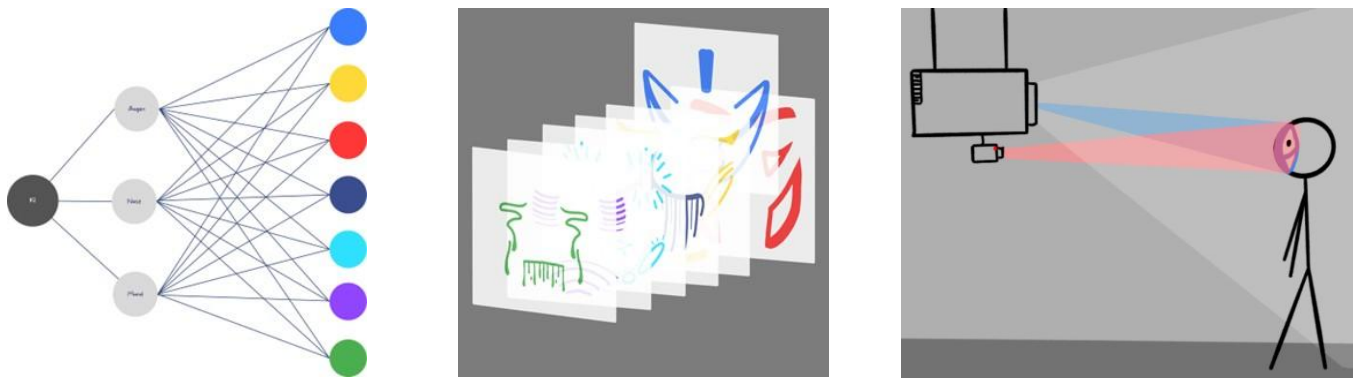


Abbildung 3: KI - Maskenerstellung - automatische Kalibrierung